

내가 설계한 미니게임 — 과제 제출 보고서

게임: 「딱 하나 이상함」

공개 주소: <https://stulss.github.io/odd-one-out/>

소스 코드: <https://github.com/stulss/odd-one-out>

프레임워크·빌드도구·이미지 파일·음원 파일 없이 순수 HTML/CSS/JavaScript로 만든 브라우저 관찰력 게임입니다.

어디로 가나요

<https://stulss.github.io/odd-one-out/>

소스 코드: <https://github.com/stulss/odd-one-out>

로컬에서 확인하려면 상위 폴더에서 아래를 실행하고 <http://localhost:8124>로 접속하세요.

(ES 모듈을 쓰므로 **index.html**을 더블클릭해서 열면 동작하지 않습니다. 반드시 서버로 여세요.)

,

```
python -u ".claude/serve.py" 8124 odd-one-out
```

,

무엇을 하나요 (3단계, 30초 이내)

1. 위 공개 주소로 접속하고 [▶ 시작하기] 를 누릅니다.
2. 격자에서 딱 하나 다른 칸을 찾아 클릭(또는 터치)합니다. 키보드로 하려면 방향키로 옮긴 뒤 `<kbd>Enter</kbd>` 를 누릅니다.
3. 시간이 0이 되어 결과 화면이 나오면 [다시 하기] 를 누릅니다.

무엇이 보이면 통과인가요

- 규칙·조작·상태가 화면에 항상 있다 — 화면 아래에 세 줄이 계속 보입니다.
- 규칙 여러 개 중 딱 하나만 다릅니다. 제한 시간 안에 찾으세요. 오답은 시간 1초 차감.
- 조작 클릭·터치 · 방향키로 이동 후 `Enter` · `P` 또는 `Esc`로 일시정지
- 상태: 진행 중 · 단계 7 · 난이도 $T=4.4$ ← 진행에 따라 바뀝니다
- 조작 1회에 화면이 바뀌고 성공/실패가 구분된다
- 다른 칸을 맞히면 즉시 다음 격자로 넘어가고 STAGE 숫자와 SCORE가 올라갑니다.
- 엉뚱한 칸을 누르면 그 칸이 빨강게 흔들리고 남은 시간이 1초 줄어듭니다. (게임은 계속됩니다)
- 시간이 0이 되면 **실패** — 시간 초과 와 도달 단계·점수가 뜨고, 정답이 어디였는지 민트색 테두리로 공개됩니다.
- 다시 시작하면 초기화된다 — [다시 하기]를 누르면 STAGE 1, SCORE 0으로 돌아갑니다. 단 최고 기록은 그대로 남습니다(보존하기로 정한 값). 타이틀 화면 아래에서 확인할 수 있습니다.
- 음소거가 작동한다 — 오른쪽 위 → 소리 를 끄면 즉시 무음이 됩니다. 다시 켜면 돌아옵니다.
- 움직임 줄이기가 작동한다 — → 움직임 줄이기 를 켜면 정답 시 확대 애니메이션과 오답 시 흔들림이 멈추고 색 점멸로 대체됩니다. (효과를 완전히 없애지 않는 이유: 맞았는지 틀렸는지 자체를 알 수 없게 되기 때문입니다.)
- 일시정지가 작동한다 — `<kbd>P</kbd>` 또는 `<kbd>Esc</kbd>` 또는 버튼을 누르면 타이머가 멈추고, 일시정지 중에는 칸을 눌러도 반응하지 않습니다. [계속하기]를 누르면 남은 시간 그대로 이어집니다.
- 같은 링크는 같은 문제가 나온다 — 결과 화면의 [결과 공유]로 나온 링크(`?c=XXXXXX` 형태)를 다른 기기나 다른 브라우저에서 열면 1단계부터 완전히 같은 문제가 나옵니다.
- 콘솔에 빨간 오류가 없다 — `<kbd>F12</kbd>` → Console 탭에 빨간 오류가 0건입니다.
- 두 해상도에서 잘리지 않는다 — 창을 1366×768, 1920×1080으로 바꿔도 가로 스크롤이 없고 격자와 하단 3줄이 모두 화면 안에 들어옵니다.

추가로 볼 수 있는 것 (선택)

- 오늘의 문제 — 타이틀 화면의 [오늘의 문제]를 누르면 그날 날짜(KST 기준)로 고정된 문제가 나옵니다. 같은 날이면 누가 언제 열어도 같은 문제입니다.
- 결과 카드 — 결과 화면에 세로형 카드 미리보기가 뜹니다. 점수뿐 아니라 마지막 문제의 격자가 정답 표시 없이 들어가 있어서, 카드를 본 사람이 직접 풀어볼 수 있습니다.
- 차이의 종류 — 정답을 맞히면 화면 가운데에 **회전 · 0.42초**처럼 어떤 차이였는지가 표시됩니다. 차이 축은 색조·밝기·크기·회전·모서리·투명도·위치·기울기·채도·두께 10종입니다.

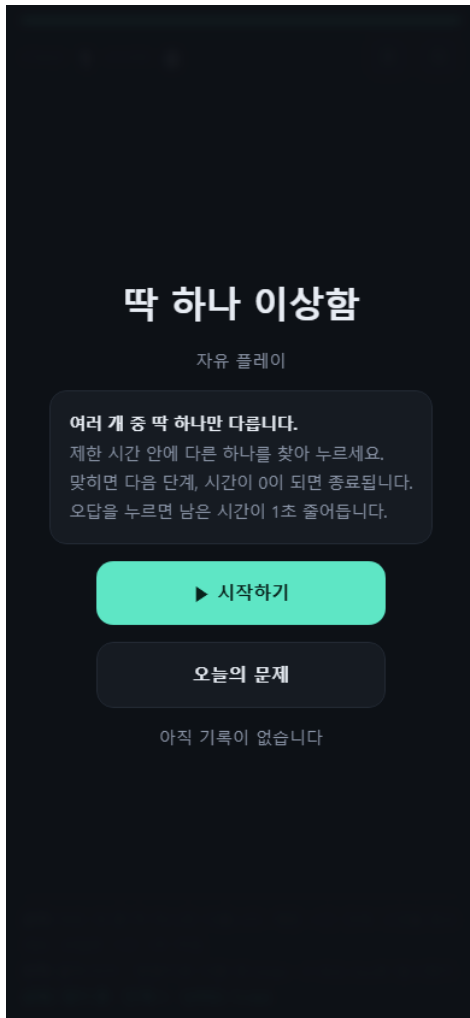
안 될 때

- 화면이 비어 있다 → `<kbd>Ctrl</kbd>+<kbd>F5</kbd>`로 강력 새로고침하세요.
- 로컬에서 열었더니 아무것도 안 나온다 → **index.html**을 더블클릭하면 동작하지 않습니다. ES 모듈은 **file://**에서 차단되므로 위의 서버 명령으로 여세요.
- 소리가 안 난다 → 브라우저는 사용자가 무언가 누르기 전에는 소리를 내지 못합니다. [시작하기]를 한 번 누른 뒤에도 안 나면 에서 소리가 꺼져 있는지, 시스템 볼륨이 0인지 확인하세요.
- 키보드를 눌러도 반응이 없다 → 버튼에 포커스가 남아 있을 수 있습니다. 격자의 아무 칸이나 한 번 클릭한 뒤 방향키를 눌러보세요.
- 게임이 저절로 멈춘 것 같다 → `<kbd>P</kbd>`나 `<kbd>Esc</kbd>`를 눌렀는지 확인하세요. 일시정지 화면이 떠 있으면 [계속하기]를 누르면 됩니다.
- 기록이 이상하다 → → [기록 초기화]를 누르면 됩니다. (저장값이 비어 있거나 깨져 있어도 게임은 기본값으로 정상 실행됩니다.)

버튼 동작 전 / 후

조작 1회에 화면 상태가 실제로 바뀌는지를 전·후 두 장으로 확인합니다. 모든 사진은 공개 주소에서 390×844 해상도로 촬영했으며, 전·후 이미지가 동일하면 촬영 실패로 간주해 자동 검사합니다.

[▶ 시작하기] 버튼

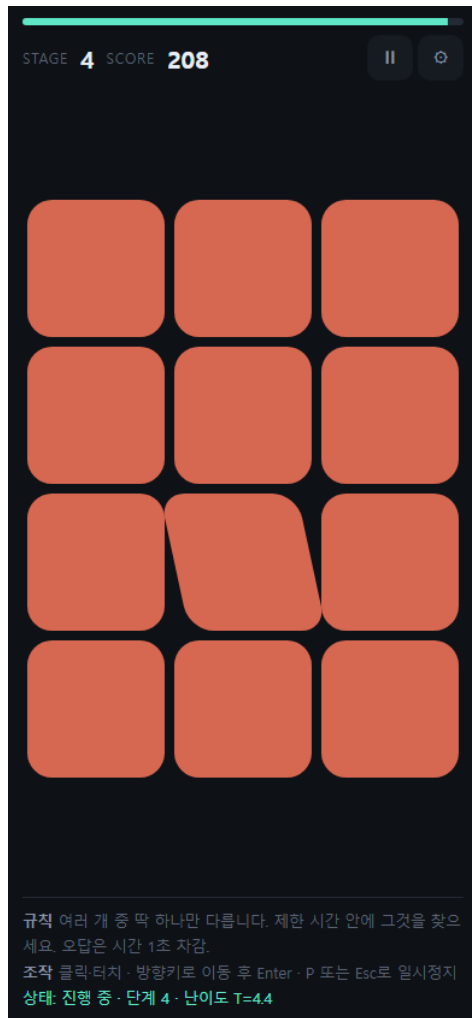


전 — 타이틀 화면(규칙 노출)

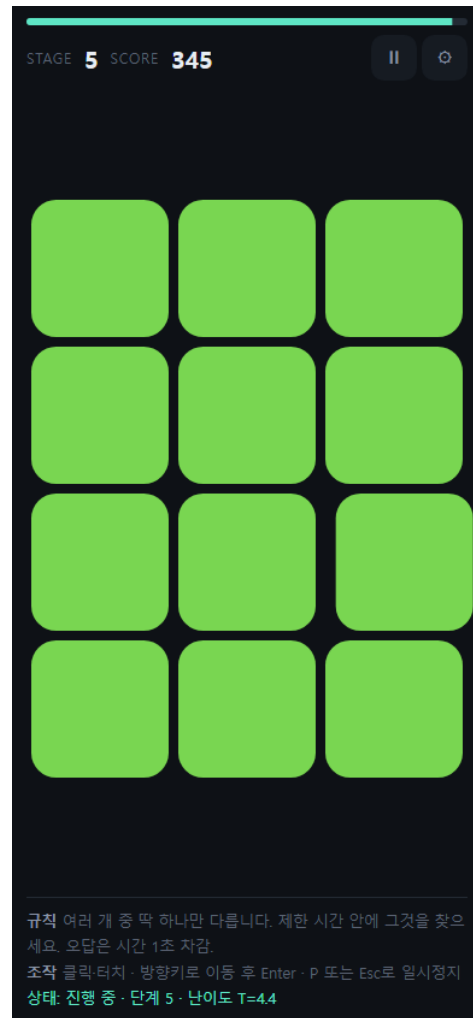


후 — 게임 시작 — 격자·타이머·점수 표시

정답 칸 선택

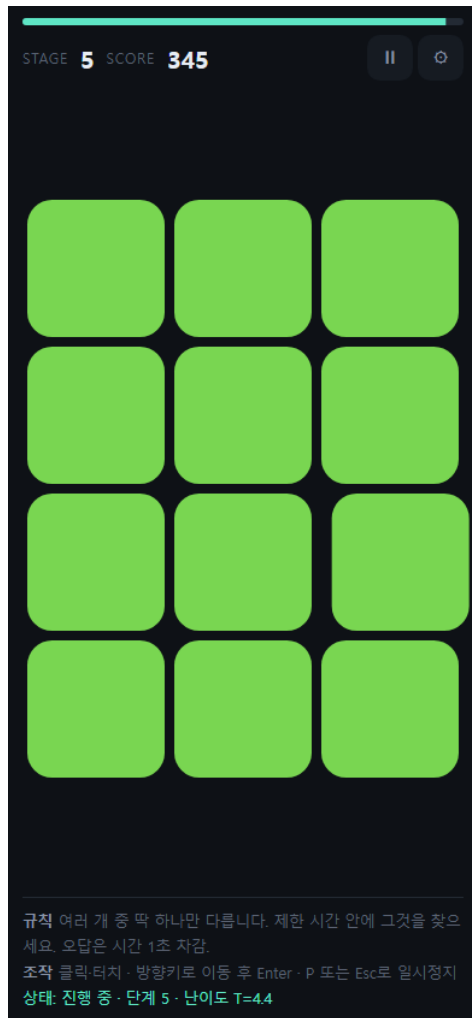


전 — STAGE N 진행 중

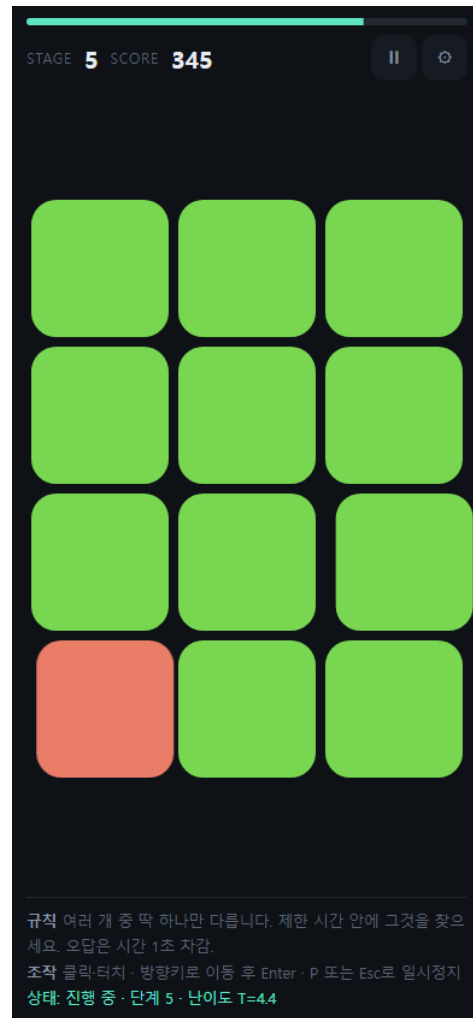


후 — STAGE 증가 + 점수 상승 + 새 격자

오답 칸 선택

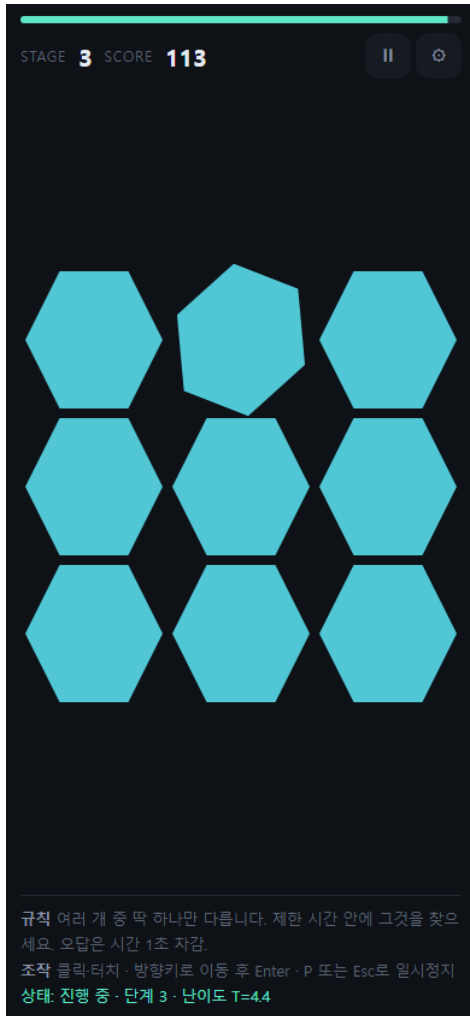


전 — 누르기 직전

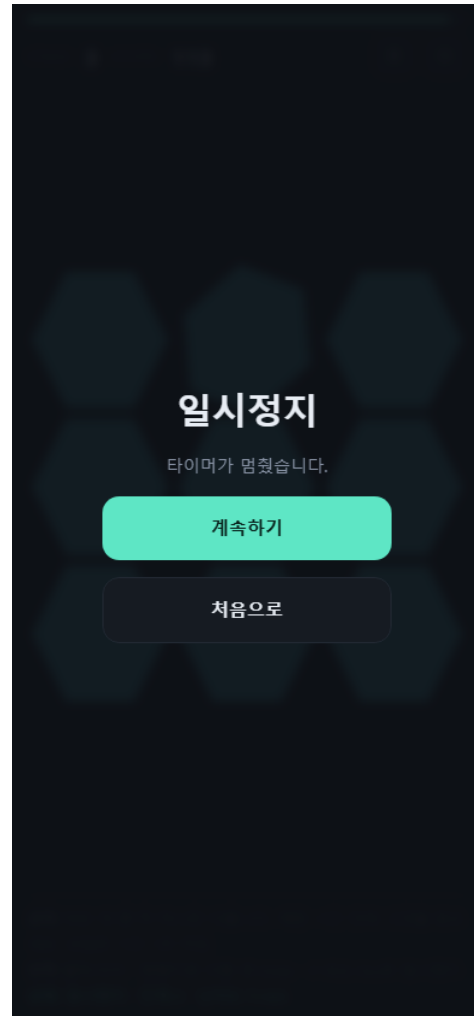


후 — 해당 칸 빨강 + 남은 시간 1초 차감

[일시정지] 버튼

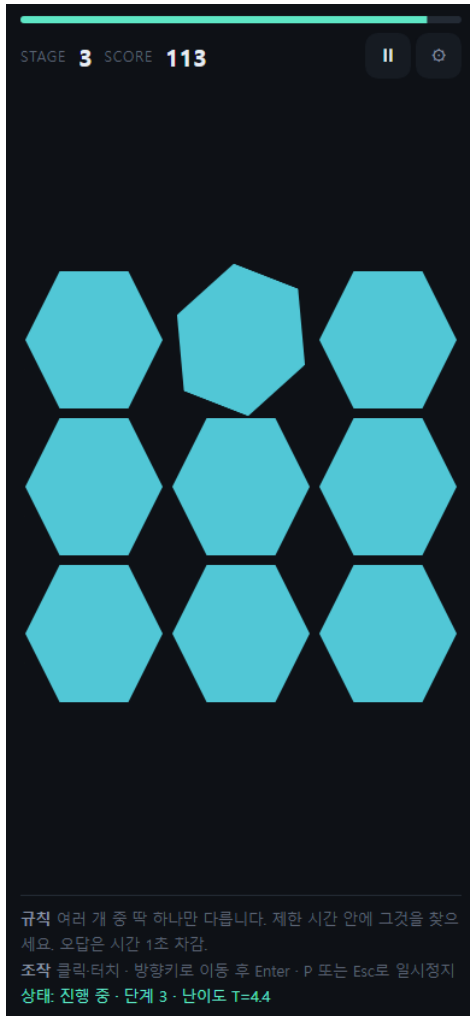


전 — 진행 중

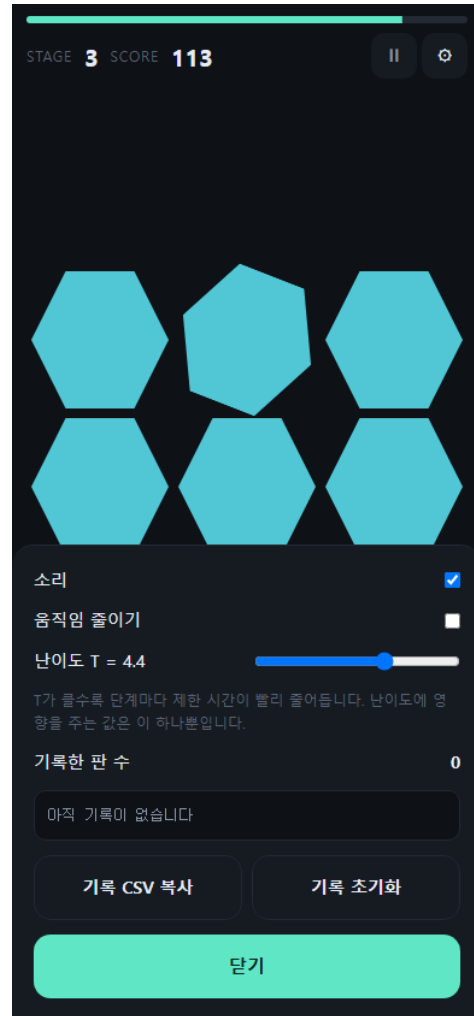


후 — 타이머 정지 + 일시정지 화면

[설정] 버튼

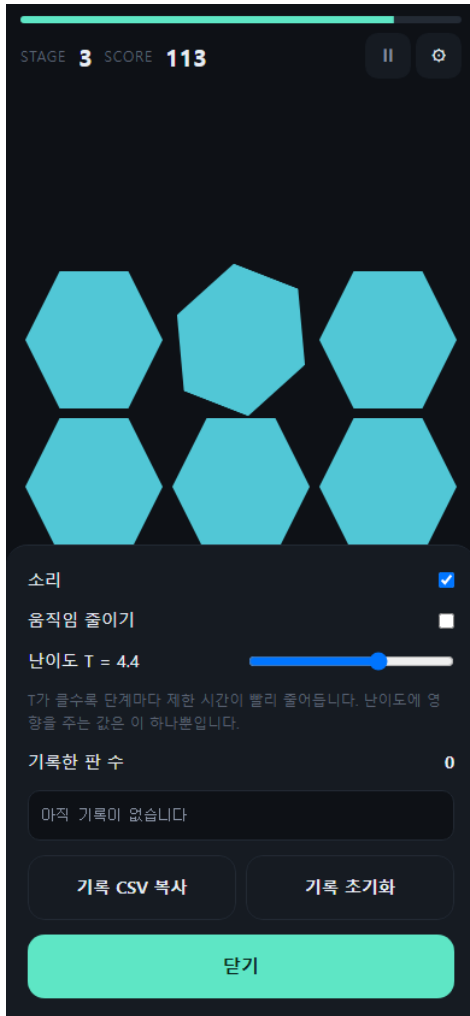


전 — 설정 열기 전

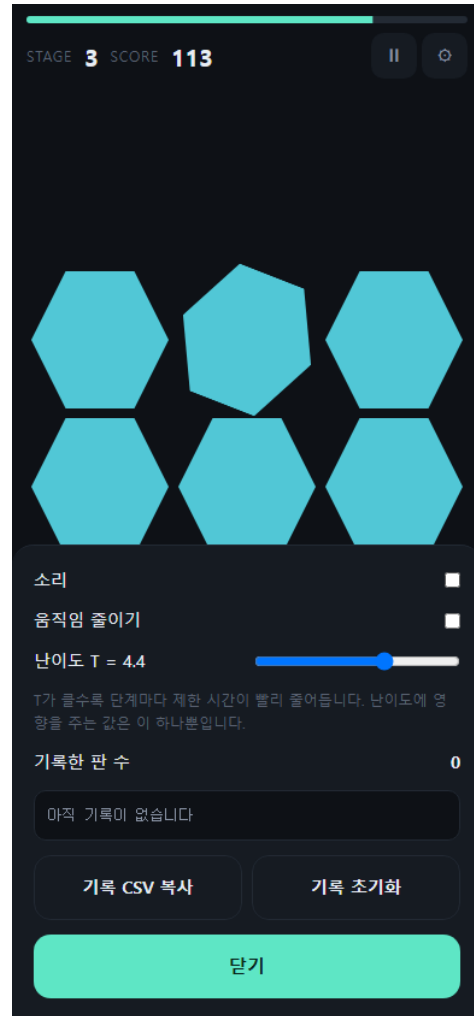


후 — 설정 시트 — 소리·움직임·난이도 T

[소리] 토글

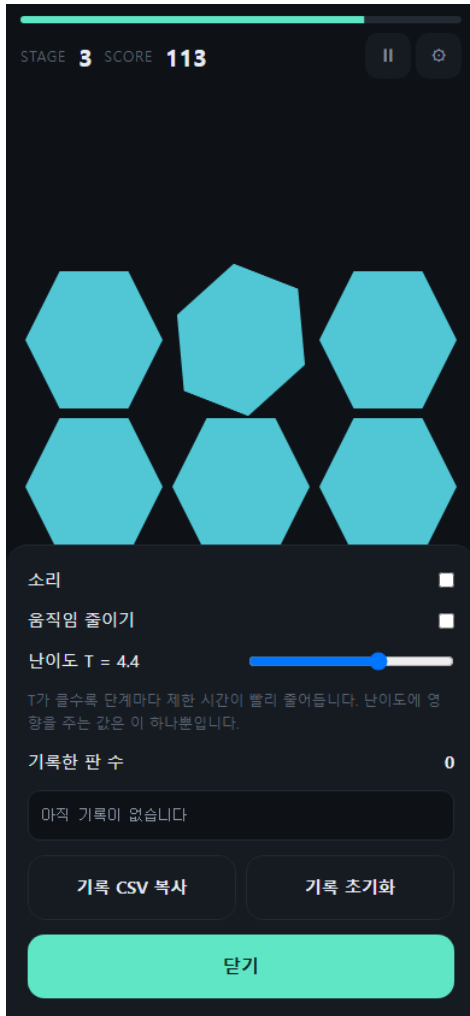


전 — 소리 켜짐

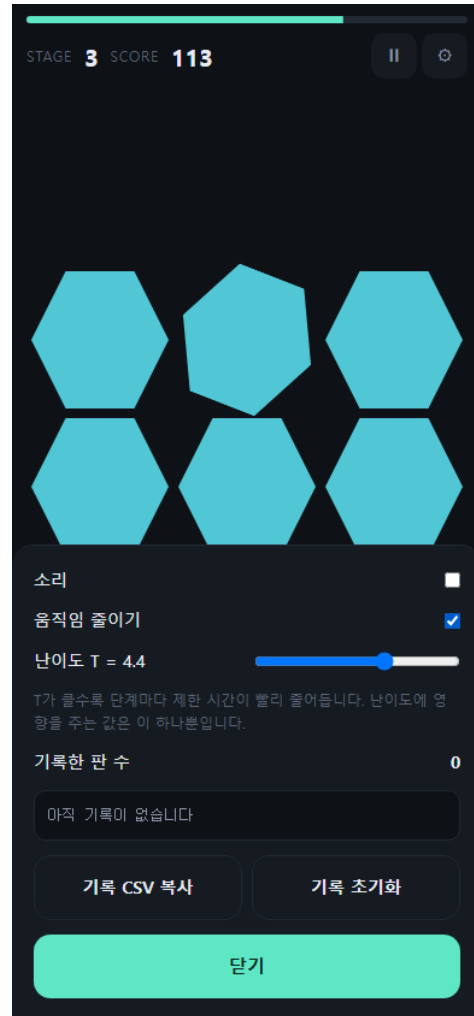


후 — 소리 꺼짐 — 즉시 무음

[움직임 줄이기] 토글

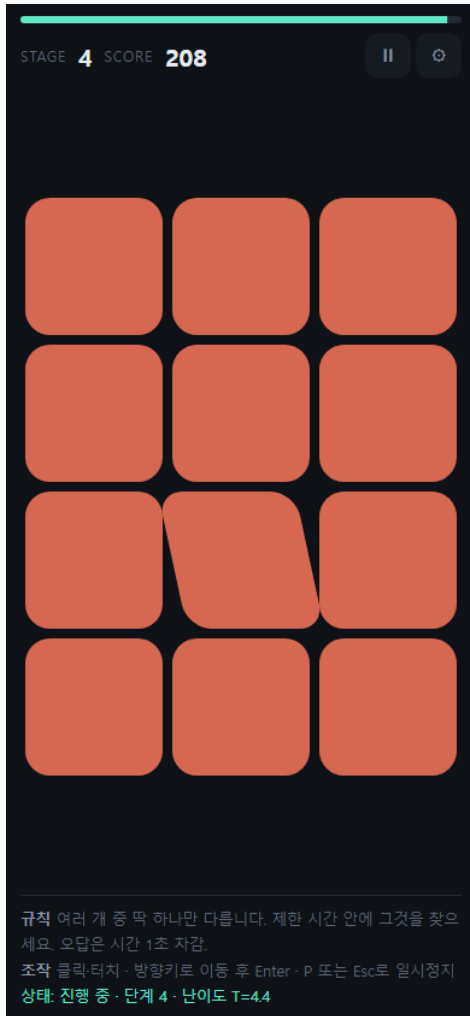


전 — 움직임 보통



후 — 움직임 줄임 — 흔들림 대신 색 점멸

시간 초과 → 종료

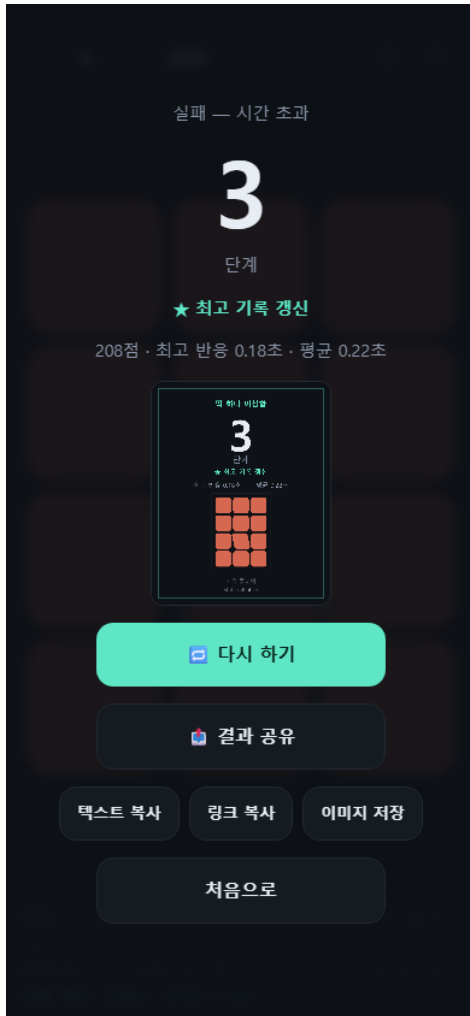


전 — 진행 중 (시간 남음)



후 — 실패 표시 + 정답 공개 + 결과 카드

[결과 공유] 버튼

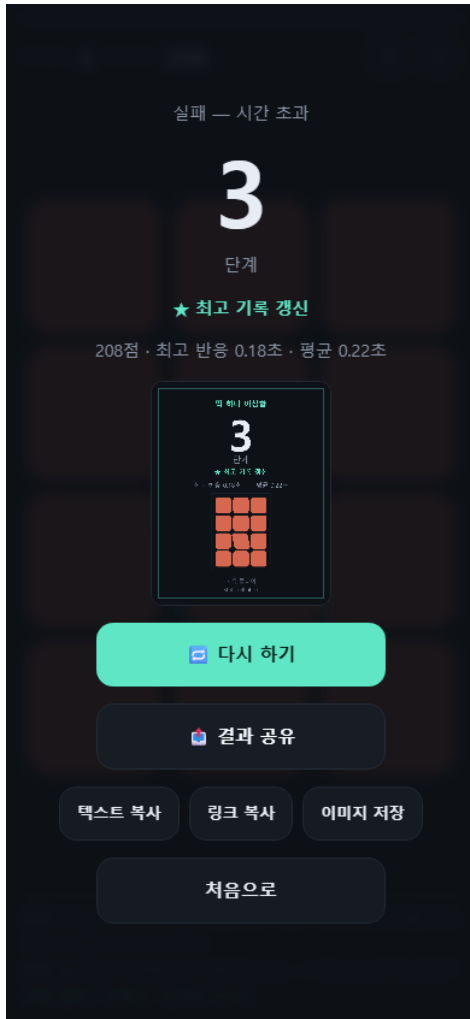


전 — 누르기 전



후 — 네이티브 공유 없음 → 복사 버튼 노출

[다시 하기] 버튼



전 — 결과 화면



후 — STAGE 1 · SCORE 0 초기화 (최고 기록 유지)

제출 전에 본인이 반드시 검토하세요. 아래는 실제 작업 기록을 바탕으로 정리한 초안입니다.

하지 않은 일을 했다고 적으면 문서 전체의 신뢰가 무너집니다.

- AI에게 맡긴 일: 게임 코드 전체(HTML/CSS/JS 13개 모듈) 작성과 검증을 맡겼다. 시드 난수(xmur3 + mulberry32) 구현, 차이 축 10종과 지각 임계값 설계, Canvas 결과 카드 생성, WebAudio 효과음 합성, localStorage 손상값 방어 코드를 AI가 썼다. 검증도 AI가 실제로 실행해 숫자를 뽑았다 — 같은 시드로 25스테이지를 두 번 생성해 완전히 일치하는지, 저장값을 6가지로 망가뜨려도 기본값으로 실행되는지, 1초에 10회 연타해도 진행이 1단계만 되는지, 일시정지 중 남은 시간이 400ms 동안 0.05초 미만으로만 변하는지, 두 해상도에서 게임판 좌표가 뷰포트 안에 들어오는지를 브라우저에서 직접 측정했다. 단계별 스크린샷 48장 촬영도 AI가 했다.
- 내가 판단한 일: 어떤 게임을 만들지, 무엇을 빼고 무엇을 만들지의 범위를 내가 정했다. 배포처를 AI가 제안한 Cloudflare Pages에서 GitHub Pages로 바꾼 것, 프로토타입을 단계별 폴더(step1~step6)로 나눠 실제 순서대로 만들게 한 것, 스크린샷을 첫화면·진행중·종료 세 상태로 찍는 규칙을 정한 것이 내 결정이었다. STEP 6을 중단하고 문서화로 넘어가라고 방향을 바꾼 것도 내 판단이었다 — 기능을 더 붙이는 것보다 지금까지 한 일을 남기는 게 먼저라고 봤다. AI가 찾아온 결함을 고칠지, 어떤 순서로 고칠지도 내가 정했다.
- AI 말을 안 들은 일: AI가 "스크린샷 촬영 정상"이라고 보고했지만 나는 직접 파일을 열어봤고, step2 스크린샷이 아직 오류 화면인 것을 발견했다. AI가 재촬영할 때 step1과 step3만 돌리고 step2를 빠뜨린 것이었다. 이걸 지적하고 나서야 전수 검사가 들어갔다. 또 AI는 난이도 최종값을 자동 플레이 데이터로 정하자고 했지만, 그 시뮬레이션은 사람의 반응 속도와 피로를 반영하지 못한다. 그 숫자를 과제의 "10회 플레이 기록"으로 내면 지어낸 데이터가 되므로 쓰지 않기로 했고, 10회씩 두 번은 내가 직접 플레이해서 채우기로 했다.

「딱 하나 이상함」(관찰력 미니게임)을 만들면서 실제로 부딪힌 문제와 해결 과정 기록입니다.

각 항목은 문제 / 시도 / 비교 / 배운 점 네 칸으로 정리했습니다.

(개인 포트폴리오의 "트리블슈팅" 섹션에 그대로 옮겨 쓸 수 있는 형식입니다.)

어떤 프로젝트인가

한 줄 소개: 여러 개 중 딱 하나 다른 것을 제한 시간 안에 찾는, 브라우저에서 바로 돌아가는 관찰력 게임.

- 격자에 비슷한 도형이 여러 개 놓이고, 그중 딱 하나만 색·크기·회전·기울기 같은 값이 미세하게 다릅니다. 제한 시간 안에 그것을 찾아 누르면 다음 단계, 시간이 0이 되면 종료.
- 난이도는 칸 수가 아니라 차이의 미묘함으로 올라갑니다. 차이 축이 10종이라 매 판 다른 종류의 관찰을 요구합니다.
- 프레임워크·빌드도구·외부 라이브러리를 하나도 쓰지 않았습니다. 순수 HTML/CSS/JavaScript(ES 모듈)로 만들었습니다.
- 이미지 파일도 0개, 음원 파일도 0개입니다. 게임 오브젝트는 전부 CSS로 그리고, 효과음은 Web Audio API로 합성합니다.
- 같은 링크를 연 사람은 완전히 동일한 문제를 봅니다. 서버 없이 시드 난수만으로 구현했습니다.

왜 이렇게 만들었나: 이 게임의 화면은 "정지된 도형 격자 + 한 번의 입력"입니다. 매 프레임 다시 그릴 것이 없습니다. 렌더 루프가 필요 없는 게임에 게임 엔진을 쓰면 엔진의 비용(번들 크기·초기화 시간)만 남고 이득이 0이라고 판단했습니다.

문제 1 — 같은 링크인데 기기마다 다른 문제가 나왔다

문제	프로토타입을 두 해상도에서 검증하다가, 격자 칸 수가 화면 크기에 따라 달라지는 것을 발견했습니다. 1280px에서는 $6 \times 8 = 48$ 칸, 375px에서는 $4 \times 6 = 24$ 칸이었습니다. 칸 수가 다르면 정답 인덱스도 달라집니다. 즉 같은 시드라도 기기마다 다른 문제가 나오고 있었습니다. 이 프로젝트의 핵심 기능인 '오늘의 문제'와 '도전장 링크'가 성립하지 않는 상태였습니다.
시도	원인은 제가 44px 터치 타겟을 지키려고 "화면이 좁으면 칸 수를 줄인다"는 규칙을 넣은 것이었습니다. 조작성을 위한 판단이었지만, 그게 시드 재현성을 통째로 깨뜨린다는 걸 그때는 몰랐습니다. 그리드를 스테이지 번호로만 결정하도록 바꾸고, 상한을 360px 기기에서도 44px가 보장되는 $4 \times 6 = 24$ 칸으로 고정했습니다. 난이도는 칸 수 대신 차이의 미묘함으로만 올리도록 했습니다.
비교	고치기 전에는 같은 시드로 뽑은 25스테이지가 기기마다 달랐고, 고친 뒤에는 25스테이지 전부 완전히 일치했습니다. 수정 규모는 코드 3줄이었습니다.
배운 점	이 결함은 발견 시점에 3줄이었지만, 공유 기능을 다 만든 뒤에 발견했다면 기능 하나를 통째로 다시 설계해야 했습니다. 되돌리기 비싼 결함일수록 일찍 찾아야 한다는 걸, 손해를 보기 전에 배웠습니다. 그리고 "조작성을 위한 좋은 규칙"이 다른 축의 요구사항을 파괴할 수 있다는 것도요.

문제 2 — 통과하던 검사가 사실 아무것도 검사하지 않고 있었다

문제	두 해상도에서 가로 넘침이 없는지 자동으로 검사하고 "넘침 없음"을 여러 번 확인했습니다. 그런데 스크린샷을 보니 게임판이 오른쪽으로 잘려 있었습니다. 검사는 통과하는데 눈으로는 넘쳐 보이는 상황이었습니다.
시도	검사 코드를 다시 봤습니다. <code>document.documentElement.scrollWidth > window.innerWidth</code> 로 재고 있었는데, CSS에 <code>html,body{ overflow-x:hidden }</code> 이 걸려 있었습니다. 넘친 내용이 잘려나가므로 <code>scrollWidth</code> 가 커지지 않습니다. 즉 이 검사는 어떤 경우에도 <code>false</code> 를 반환하는, 실패할 수 없는 검사였습니다. 게임판의 실제 좌표를 뷰포트 경계와 직접 비교하는 방식(<code>board.getBoundingClientRect().right > innerWidth</code>)으로 바꿨습니다.
비교	바꾼 검사로 390×844에서 11스테이지 전 구간을 다시 잴더니, 게임판은 16px~374px에 있었고(뷰포트 390px) 실제로는 넘치지 않았습니다. 스크린샷 쪽이 잘못된 것이었습니다(문제 4). 하지만 그 사실을 확인할 수 있게 된 것 자체가 검사를 고친 덕분입니다.
배운 점	초록불을 믿기 전에 그 검사가 빨간불이 될 수 있는 검사인지 먼저 확인해야 합니다. 항상 통과하는 검사는 검사가 아니라 장식입니다. 이후로는 검증 코드를 짤 때 "이 검사가 실패하는 경우를 하나 만들어볼 수 있나"를 먼저 생각하게 됐습니다.

문제 3 — 스크린샷 12장이 전부 오류 화면이었는데 스크립트는 전부 "OK"였다

문제	해상도별 증거 스크린샷을 남기려고 헤드리스 브라우저로 12장을 찍었습니다. 스크립트는 12줄 모두 OK를 출력했습니다. 그런데 파일 크기 목록을 보니 단계가 다른데 크기가 완전히 같았습니다. 열어보니 12장 전부 "사이트에 연결할 수 없음(ERR_CONNECTION_REFUSED)" 오류 화면이었습니다.
시도	원인은 두 겹이었습니다. ① 실행 환경이 네트워크 샌드박스라 로컬 개발 서버에 접근할 수 없었습니다. ② 성공 판정을 "파일이 2KB 이상 생성됐는가"로 했기 때문에, 오류 화면 PNG도 성공으로 집계됐습니다. <code>file://</code> 프로토콜로 전환해 서버 없이 촬영하도록 바꾸고, 스크립트에 이미지 해시 중복 검사를 넣었습니다. 서로 다른 화면인데 이미지가 같으면 촬영 실패이므로 경고를 띄웁니다.
비교	고치기 전에는 12장 전부 오류 화면인데 OK 12줄. 고친 뒤에는 48장 / 고유 이미지 48종 / 중복 0이 출력되고, 실제로 열어봐도 게임 화면이 나옵니다.
배운 점	"스크립트가 성공했다"와 "결과물이 옳다"는 다른 문장입니다. 그리고 성공 판정 기준을 느슨하게 잡으면 자동화는 틀린 결과를 빠르게 대량으로 만들어냅니다. 산출물은 결국 열어봐야 하고, 자동 검사에는 "이게 실패했다면 어떤 흔적이 남는가"를 넣어야 했습니다.

문제 4 — 고쳤다고 하고 한 폴더를 빠뜨렸다 (사용자가 발견)

문제	문제 3을 고친 뒤 "스크린샷 정상"이라고 보고했는데, 사용자가 "스텝2 스크린샷은 아직도 오류네" 라고 알려줬습니다. 확인해보니 사실이었습니다.
시도	재촬영할 때 step1 과 step3 만 실행하고 step2 를 빼먹었습니다. 그래서 step2 폴더에만 예전 오류 화면이 그대로 남아 있었습니다. 저는 "고쳤다"는 사실만 보고 전체를 다시 확인하지 않았습니다. step2를 다시 찍고, 이번에는 모든 PNG의 해시를 비교해 오류 화면 시그니처가 남아 있지 않은지 전수 확인했습니다.
비교	사용자가 알려주기 전: 12장 중 4장이 오류 화면인데 "정상"이라고 보고된 상태. 알려준 뒤: 전수 검사를 거쳐 중복 0 확인. 이후 촬영 스크립트 자체에 중복 검사를 넣어 같은 실수가 조용히 지나가지 않게 만들었습니다.
배운 점	문제 3에서 "결과물을 열어봐야 한다"를 배웠는데, 정작 고친 뒤에 전부 다시 확인하지 않았습니다. 부분 수정 후 전체 재검증을 건너뛰는 것이 같은 실수의 다른 얼굴이었습니다. 그리고 이 결함은 제가 아니라 실제로 파일을 열어본 사람이 찾았습니다.

문제 5 — 촬영 도구가 지정한 해상도를 지키지 않았다

문제	오류 화면 문제를 고친 뒤에도, 360×640과 390×844 스크린샷에서는 게임판이 오른쪽으로 잘려 보였습니다. 그런데 실제 브라우저에서 좌표를 재보면(문제 2의 고친 검사) 넘치지 않았습니다. 증거와 측정이 서로 다른 말을 하고 있었습니다.
시도	헤드리스 브라우저의 <code>--window-size</code> 가 실제 뷰포트를 그대로 지정해주지 않고, 더 넓은 폭으로 레이아웃한 뒤 이미지를 지정 크기로 잘라내고 있었습니다. <code>--force-device-scale-factor=1</code> 을 줘도 고쳐지지 않았습니다. 뷰포트를 정확히 지정할 수 있는 Playwright로 촬영 도구를 교체했습니다. 부수적으로, 페이지 안에서 JS를 실행할 수 있게 되어 첫 화면 / 진행 중 / 종료 화면 세 가지 상태를 각각 찍을 수 있게 되었습니다.
비교	교체 전: 작은 해상도 스크린샷이 잘려서 증거로 쓸 수 없었고, 상태도 첫 화면 하나만 찍혔습니다. 교체 후: 360×640에서 4×4 격자가 온전히 들어가고 일시정지 버튼·규칙 3줄이 모두 보이며, 단계당 3상태 × 4해상도 = 12장이 정확히 나옵니다.
배운 점	측정값 두 개가 서로 다른 말을 할 때는 둘 중 하나가 아니라 도구를 의심해야 하는 경우가 있습니다. 여기서는 코드가 아니라 촬영 도구가 틀렸습니다. 그리고 도구를 바꾸니 원래 하려던 것(3상태 촬영)까지 가능해졌습니다.

문제 6 — 난이도 손잡이가 플레이어가 보는 구간에서는 거의 움직이지 않았다

문제	난이도가 스테이지마다 오르도록 만들어놓고 실제 값을 뽑아봤습니다. 차이(회전 각도)가 1단계 20.0°, 10단계 15.0°였습니다. 10판을 하는 동안 25%밖에 좁혀지지 않았습니다. 반면 15~25단계에서는 10.4° → 3.1°로 급격히 어려워졌습니다.
시도	원인은 난이도 곡선에 smoothstep ($t^2(3-2t)$)을 쓴 것이었습니다. 이 함수는 양 끝이 완만하고 가운데가 가파릅니다. 그런데 이 게임은 제한 시간이 짧아 대부분 8~15단계에서 끝납니다. 즉 플레이어가 실제로 보는 구간이 하필 평평한 구간이었습니다. 곡선을 선형으로 바꿨습니다.
비교	변경 전(smoothstep) 1/5/10/15/20단계에서 20.0° / 18.8° / 15.0° / 10.4° / 5.9°. 변경 후(선형) 20.0° / 17.3° / 13.9° / 10.5° / 7.1°. 초반 10판의 변화량이 5.0° → 6.1°로 늘고, 판마다 고르게 좁혀집니다.
배운 점	곡선을 고를 때 "수학적으로 부드러운가"가 아니라 "플레이어가 실제로 머무는 구간이 어디인가"를 먼저 봐야 했습니다. 다만 이 변경은 곡선 모양만 고친 것이고, 최종값은 사람이 직접 10판씩 해봐야 정할 수 있습니다. 자동 플레이로 만든 숫자는 사람의 반응·피로를 반영하지 못하므로 쓰지 않기로 했습니다.

이 프로젝트가 남긴 것

- 엔진을 안 쓰는 것도 기술 선택이라는 걸 배웠습니다. "무엇을 쓸까"보다 "이 게임이 실제로 무엇을 필요로 하는가"를 먼저 물었더니 답이 라이브러리 0개였고, 그 덕에 전송 크기가 80KB로 끝났습니다.
- 자동 검증을 의심하는 연습을 했습니다. 항상 통과하던 검사(문제 2), 오류를 성공으로 집계하던 스크립트(문제 3), 지정을 무시하던 촬영 도구(문제 5) — 세 번 다 "초록불"이었지만 아무것도 증명하지 않고 있었습니다.
- 부분 수정 뒤 전체 재검증의 중요성을 실수로 배웠습니다(문제 4). 고쳤다는 사실이 전부 고쳐졌다는 뜻은 아니었습니다.
- 감으로 상수를 바꾸지 않는 습관이 붙었습니다. 난이도가 평탄하다는 걸 알면서도 감으로 고치지 않고 실제 값을 뽑아 표로 비교한 뒤에 바꿨고, 최종값은 사람이 직접 재야 한다는 것도 남겨두었습니다.

1. 개요 및 역할

항목	내용
프로젝트명	「딱 하나 이상함」 (Odd One Out)
기간	2026-08-18 ~ 2026-08-19 (2일) — 당초 MVP 계획은 7일, 실제로는 2일에 코어 완성
팀 구성	1인 (기획 · 설계 · 개발 · QA · 배포 전 과정)
협업 도구	Claude (Claude Code) — 코드 작성 및 검증 자동화에 사용
결과물	웹 미니게임 1종 + 설계 문서 6종 + 단계별 구현 기록
저장소 구성	odd-one-out/ (본 빌드) · prototype/ (단계별 구현 기록)

역할 분담 — 사람과 AI

포트폴리오에 "기여도 100%"라고 적는 것보다, 무엇을 스스로 판단했는지를 적는 편이 신뢰를 얻습니다. 아래 표는 실제 작업 분담이며, 제출 전 본인 기준으로 검토·수정하시기 바랍니다.

영역	사람이 결정	AI가 수행
게임 규칙 · 실패 조건	● 최종 결정	초안 제시
기술 스택 선택	● 승인	비교안·근거 작성
배포처 변경 (Cloudflare → GitHub Pages)	● AI 제안을 반려했고 변경	초기 제안
범위 결정 (무엇을 빼는가)	● 최종 결정	제거 후보 제시
코드 구현	리뷰	● 작성
검증 시나리오 실행	리뷰	● 자동화 실행
난이도 상수 최종값	○ 미정 — 플레이 데이터 수집 후 본인이 결정	후보 3안 제시

위 표의 ●/○ 표시는 실제 작업에 맞게 반드시 본인이 재검토하세요.

하지 않은 일을 했다고 적는 순간 포트폴리오 전체의 신뢰가 무너집니다.

2. 문제 정의

무료 웹 미니게임에서 이탈이 발생하는 지점은 재미가 아니라 진입입니다.

- 게임 엔진 번들을 받는 동안 이탈
- 규칙을 설명하는 튜토리얼을 읽는 동안 이탈
- 로그인·회원가입 화면에서 이탈

그래서 목표를 "예쁜 게임"이 아니라 "3초 안에 첫 플레이"로 잡았고, 이 기준과 충돌하는 기능은 전부 잘라냈습니다.

설정한 성공 기준 (측정 가능한 형태로)

1. 첫 화면에서 플레이까지 3초 이내
2. 서버 비용 0원 (로그인·DB·API 없음)
3. 같은 링크를 연 사람은 완전히 동일한 문제를 본다
4. 규칙·조작·현재 상태가 화면에 항상 보인다
5. 콘솔 오류 0건

3. 기술 선택과 이유

결론: 게임 엔진을 쓰지 않고 Vanilla JS + DOM으로 만들었습니다

일반적으로 웹 게임에는 Phaser나 Unity WebGL을 씁니다. 쓰지 않기로 한 이유는 하나입니다.

이 게임의 화면은 "정지된 도형 격자 + 한 번의 입력"이다.

매 프레임 다시 그릴 것이 없다. 스크롤도, 물리도, 카메라도, 스프라이트 애니메이션도 없다.

렌더 루프가 필요 없는 게임에 게임 엔진을 쓰면 엔진의 비용만 남고 이득은 0이다.

후보	최소 전송 크기	첫 플레이까지	판단
Vanilla + DOM	약 25KB	약 0.8초	채택
PixiJS	약 140KB	약 1.8초	과함
Phaser 3	약 450KB	약 3.0초	목표 위반
Unity WebGL	15MB+	약 15초	치명적

※ 위 비교표의 크기·시간은 일반적으로 알려진 수치를 근거로 한 추정입니다.

실제로 측정한 값은 본 프로젝트의 산출물 크기(아래 5장)뿐입니다.

부수적으로 얻은 것

- 터치 히트 판정을 브라우저가 처리 → 좌표 계산·DPR 보정 코드가 아예 필요 없음
- 키보드 포커스·접근성이 기본 제공 → 마우스 없이 완주 가능한 구조를 공짜로 확보
- 빌드 도구 없음 → 저장 후 새로고침이 곧 확인

이미지 파일을 0개로 만든 이유

이 게임의 재미는 "미묘한 차이" 입니다. 차이를 1px·3° 단위로 정확히 제어해야 난이도 곡선이 성립합니다.

래스터 이미지는 축소·DPR 보정 과정에서 차이가 뭉개지거나(너무 쉬움) 안티에일리어싱 노이즈가

생깁니다(불공정한 실패). 벡터/CSS만이 차이를 수치로 보증하므로, 게임 오브젝트를 전부 CSS로 생성했습니다.

4. 핵심 문제 해결 — Before / After

문제 ① 같은 링크인데 기기마다 다른 문제가 나왔다 (치명적)

Before 그리드 칸 수를 화면 크기에 맞춰 계산했습니다. 44px 터치 타겟을 지키려는 의도였습니다. 그 결과 같은 시드라도 기기마다 문제가 달라졌습니다.

뷰포트	그리드	칸 수
1280px	6×8	48칸

뷰포트	그리드	칸 수
375px	4×6	24칸

정답 인덱스까지 달라지므로, 프로젝트의 핵심 기능인 '오늘의 문제'와 '도전장 링크'가 성립하지 않는 상태였습니다.

어떻게 발견했는가 Day 1 프로토타입을 두 해상도에서 자동 검증하다가, 그리드 크기가 뷰포트에 따라 달라지는 것을 확인했습니다. 기능을 다 만든 뒤였다면 원인 추적에만 반나절이 들었을 문제입니다.

After 그리드를 스테이지 번호로만 결정하도록 바꾸고, 상한을 360px 기기에서도 44px가 보장되는 4×6=24칸으로 고정했습니다. 난이도는 칸 수가 아니라 차이의 미묘함(delta) 으로 올립니다. 원래 설계 철학과도 일치합니다.

항목	Before	After
같은 시드 25스테이지 재현	기기마다 다름	완전 일치 (측정)
그리드 결정 요인	화면 크기	스테이지 번호
수정 규모	—	코드 3줄

교훈: 되돌리기 비싼 결함일수록 일찍 찾아야 한다. 이 결함은 발견 시점에 3줄이었고, 배포 후였다면 기능 하나를 통째로 다시 설계해야 했다.

문제 ② 검사 자체가 틀려 있었다

Before 가로 넘침을 이렇게 검사했습니다.

```
document.documentElement.scrollWidth > window.innerWidth
```

CSS에 **html,body{ overflow-x:hidden }**이 걸려 있어 실제로 넘쳐도 항상 **false**가 나옵니다. 즉 "넘침 없음"이라는 결과는 아무것도 증명하지 못하는 값이었습니다.

After 게임판의 실제 좌표를 뷰포트 경계와 직접 비교하도록 바꿨습니다.

```
board.getBoundingClientRect().right > window.innerWidth
```

재측정 결과 (390×844, 11스테이지 전 구간)

항목	결과
게임판 좌우 위치	16px ~ 374px (뷰포트 390px)
좌·우 넘침	11스테이지 전부 없음
최소 셀 크기 (최악 격자 4×6)	83px
일시정지 버튼 · 하단 3줄	화면 안에 있음

교훈: 통과한 테스트를 믿기 전에 그 테스트가 실패할 수 있는 테스트인지 먼저 확인해야 한다.

항상 통과하는 검사는 검사가 아니다.

문제 ③ 스크린샷 증거가 전부 오류 화면이었다

Before 헤드리스 Chrome으로 해상도별 스크린샷 12장을 저장했습니다. 스크립트는 전부 **OK**를 출력했습니다. 그런데 파일 크기가 단계별로 완전히 같았습니다. 이상해서 열어보니 12장 전부 "사이트에 연결할 수 없음(ERR_CONNECTION_REFUSED)" 화면이었습니다.

원인은 실행 환경이 네트워크 샌드박스여서 로컬 개발 서버에 접근할 수 없었던 것이고, 스크립트는 "파일이 2KB 이상 생성됨"만 확인했기 때문에 오류 화면도 성공으로 집계했습니다.

After **file://** 프로토콜 + **--allow-file-access-from-files**로 전환해 재촬영하고, 실제 이미지를 눈으로 확인했습니다.

해상도	상태
1366×768 · 1920×1080	정확 (게이지·점수·버튼·격자·규칙 3줄 모두 정상)
360×640 · 390×844	신뢰 불가 — 헤더리스가 지정보다 넓은 뷰포트로 그린 뒤 잘라내 넘친 것처럼 보임

작은 해상도는 실제 브라우저 측정(위 문제 ②)으로 대체 검증했고, 스크린샷 방식 개선안 3가지를 문서에 남겼습니다.

교훈: "스크립트가 성공했다"는 것과 "결과물이 옳다"는 것은 다른 문장이다.

산출물은 반드시 열어봐야 한다.

5. 수치로 본 결과

실제로 측정한 값

항목	측정값	확인 방법
소스 전체 크기 (압축 전)	80KB (src/ 68KB + index.html 12KB)	파일 시스템
외부 라이브러리	0개	package.json 자체가 없음
이미지 파일	0개 (전부 CSS 생성)	파일 시스템
오디오 파일	0개 (WebAudio 합성)	파일 시스템
시드 재현성	25스테이지 완전 일치	자동 검증
저장값 손상 복구	6중 전부 기본값 복구 (빈값/" abc "/누락/타입오류/배열/정상)	자동 검증
연속 입력 10회 → 진행	정확히 1단계	자동 검증
일시정지 중 시간 오차	0.05초 미만 (400ms 관찰)	자동 검증
최소 셀 크기 (최악 격자)	360×640에서 67px / 390×844에서 83px (기준 44px)	실측
차이 축 종류	10종 전부 출현 확인	자동 검증
콘솔 오류	0건	DevTools

배포 후 라이브 실측 (<https://stulss.github.io/odd-one-out/>)

항목	목표	실측	판정
전송 크기 (gzip)	100KB 이하	26.3KB (13개 요청)	목표의 26%
페이지 로드 완료	3초 이내	1.5초	
하위 경로(/odd-one-out/) 모듈 로딩	정상	13개 모듈 전부 200	
공유 링크가 하위 경로 유지	유지	.../odd-one-out/?c=W3AQDR&s=4	

항목	목표	실측	판정
같은 링크 = 같은 문제	동일	새로 두 번 열어 5스태이지 완전 일치	
콘솔 오류	0건	0건	

아직 측정하지 못한 값

항목	목표	상태
Lighthouse 성능 점수	95+	미측정
10분 연속 실행	누수 0	60초-301스태이지까지만 실측 (노드 증가 0, 힙 감소)
난이도 최종값	—	사람이 직접 10판+10판 필요

포트폴리오에 미측정 항목을 성과처럼 적지 마세요. 측정한 것만 성과입니다.

6. 아키텍처

```
index.html 크리티컬 CSS 인라인 · 5화면 · 규칙 3줄 상시 표시
├── src/ (13개 모듈, 빌드 도구 없음)
├── game.js 상태 머신 TITLE → PLAY PAUSED → RESULT
├── rng.js xmr3 + mulberry32 (시드 재현성)
├── stage.js 스테이지 생성 + 공정성 검증 + 차이 축 10종
├── render.js DOM 그리드 (노드 재사용, 생성/삭제 0)
├── input.js 포인터 + 키보드, 연속입력 잠금
├── timer.js rAF 루프 1개, 일시정지·포커스 복귀 보정
├── save.js localStorage + 손상 복구
├── daily.js 오늘의 문제(KST 고정) + 도전장 코드
├── card.js Canvas 결과 카드 1080×1350
├── share.js Web Share L2 → L1 → 폴백 3종
├── audio.js WebAudio 합성음
└── logger.js 플레이 기록 + CSV + 중앙값 요약
```

설계에서 지킨 3가지 원칙

1. rAF 루프는 앱 전체에 1개만 스테이지마다 루프를 만들면 오래 커뒀을 때 루프가 누적됩니다. 10분 실행 안정성의 유일한 원인이라 처음부터 막았습니다.
2. DOM 노드는 최초 1회만 생성 셀 24개를 미리 만들어 두고 계속 재사용합니다. 스테이지 전환 시 노드 생성/삭제가 0입니다.
3. 현재 판 상태는 저장하지 않는다 점수·스테이지·타이머를 localStorage에 넣지 않으므로, 새 게임에서 자동으로 초기화됩니다. "초기화 로직"을 따로 만들면 반드시 빠뜨리는 항목이 생깁니다. 저장하지 않는 것이 가장 확실한 초기화입니다.

7. 검증 방법

기능마다 브라우저에서 자동 시나리오를 실행해 실제 DOM 좌표와 상태값을 측정했습니다. "될 것 같다"가 아니라 숫자로 확인한 뒤 다음 단계로 넘어갔습니다.

- 시드 재현성: 같은 키로 25스테이지를 두 번 생성해 JSON 비교

- 저장 손상: localStorage에 6종의 잘못된 값을 넣고 각각 실행
- 연속 입력: 1초에 10회 입력 후 진행 단계 수 확인
- 레이아웃: 4개 해상도에서 게임판 좌표를 뷰포트 경계와 비교
- 일시정지: 400ms 관찰하며 남은 시간 변화량 측정

prototype/에는 STEP 1~6을 한 단계씩 실제로 만들고 검증한 기록이 남아 있습니다. 각 단계 폴더는 그 시점의 완전한 실행본이며, 검증 결과와 판단 근거를 **STEPS.md**에 기록했습니다.

8. 의도적으로 만들지 않은 것

기능을 더하는 것보다 완성하는 것을 우선했습니다.

제외	이유
회원가입 · 로그인	서버 비용 발생 + 이탈률 최대 요인
글로벌 랭킹	서버 + 어뷰징 대응 비용. 도전장 링크가 같은 욕구를 0원으로 해결
3D · Meshy 에셋	로딩 예산의 3~10배 초과
BGM	용량 + 저작권 + 모바일 자동재생 차단
웹폰트	FOIT/FOUT + 수백 KB
프레임워크 · 번들러	이 규모에서는 손손실

9. 현재 한계와 다음 단계

정직하게 적습니다. 아래는 아직 끝나지 않은 항목입니다.

#	항목	상태
1	배포	완료 — https://stulss.github.io/odd-one-out/
2	난이도 초반 구간	1~10스테이지가 거의 평탄 (차이 17.1° → 16.7°). 발견했으나 감으로 고치지 않고 플레이 데이터 수집 후 결정하기로 보류
3	작은 해상도 스크린샷	헤드리스 캡처 신뢰 불가. Playwright 도입 필요
4	minigame_test STEP 1~6	완료 (난이도 최종값만 사람 몫)
5	세계관 · 캐릭터 대사	미착수 (게임 동작에는 영향 없음)

10. 이 프로젝트에서 배운 것

- 엔진을 안 쓰는 것도 기술 선택이다. "무엇을 쓸까"보다 "이 게임이 실제로 무엇을 필요로 하는가"를 먼저 물었더니, 답이 라이브러리 0개였습니다.
- 결함은 발견 시점에 따라 가격이 다르다. 시드/그리드 결함은 프로토타입 단계에서 3줄이었지만, 배포 후였다면 기능 하나를 다시 설계해야 했습니다.

3. 통과한 테스트를 의심해야 한다. **overflow-x:hidden** 때문에 항상 통과하던 검사, 오류 화면을 성공으로 집계하던 스크립트 — 둘 다 "초록불"이었지만 아무것도 증명하지 못하고 있었습니다.
4. 감으로 상수를 바꾸지 않는다. 난이도가 평탄하다는 것을 알면서도 고치지 않았습니다. 근거 없이 바꾼 값은 다음 사람이 되돌릴 수 없습니다. 플레이 데이터를 모은 뒤 결정하기로 하고, 후보 3안과 판단 기준을 문서에 남겼습니다.

참고 문서

문서	내용
docs/00_과제_요구사항_매핑.md	요구사항 ↔ 설계 1:1 대응
docs/01_게임기획.md	기술 스택 · UI · 시스템 · 알고리즘 전체 설계
docs/05_배포.md	배포처 비교와 변경 근거
작업내역_체크리스트.md	진행 상황 · 결정 기록 18건 · 작업 로그
prototype/STEPS.md	단계별 구현과 검증 기록